Utrecht University

Replication in Economics

Non-exhaustive tips

Vita Titl

Outline 1

1. Background

2. Repository Structure

3. The Scalars Pattern

4. End-to-End Workflow

What I Saw and/or Did in the Past

- Created v12, v13, _final, and then dates and similar
- Copy pasting and manually adjusting tables (e.g., from outreg2)
- Copy pasting findings – particular numbers – from tables
- No comments, no documentation of which file produces what

This is easy at first, but gets costly when you need to make adjustment to the code (after getting comments from reviewers) or even worse when you need to submit a replication package.

Policies of Journals

Background

Most top journals in economics (and management) require a replication package (top 5s, most top field journals, Management Science, Marketing Science etc.)

Exceptions: confidential administrative data, proprietary datasets; BUT you may still be required to show replication and you post the codes

Details: `https://aeadataeditor.github.io`

Basic requirements

- The code you provide processes raw data, runs all the analysis, and replicates every table and/or figure in the paper.
- You should provide a master script that starts the whole process and the user just waits.
- A readme file needs to provided, giving details of how to replicate the paper
  - The file explains what this replication package does
  - The folder structure
  - List of figures/tables and which code generates it
  - Example config file if needed
  - Requirements (yaml file for Conda environment or requirements.txt for python or a list of Stata packages with versions and sources if not ssc)
  - Memory and runtime requirement

How To Solve This?

- From the beginning, use Git (VITA SHOWS).
- Set the repository well (see later).
- Automate everything possible and have it ready like you were submitting a replication package tomorrow.
- Replicate everything and test whether it works (ideally let co-authors do that).

Outline 2

1. Background
2. Repository Structure
3. The Scalars Pattern
4. End-to-End Workflow

The ideal folder layout

What you need

- One **entry-point**: `code/make.sh`
- An **output** folder that analysis scripts write into
- A **writing** folder with its own `output/` sub-folder that LaTeX reads from
- A sync script: `refresh_output.sh`

Folder tree

```
project/
  code/
    make.sh
    analysis/
    scalars/
  data/
  dataLocal/
  output/
    logs/
  writing/paper/
    output/
    main.tex
  refresh_output.sh
```

One entry-point: `make.sh`

A reviewer (or your future self) should be able to run **one command** and reproduce the entire paper:

```
# -- Rerunning the project -------------------------
cd code
./main.sh

# -- Sync outputs to the writing folder --------------------
cd ..
../refresh_output.sh

# -- Compile paper ------------------------------------------
cd ../writing/paper && pdflatex main.tex
```

(VITA RUNS)

Key idea: every table, figure, and inline number is regenerated automatically before LaTeX runs.

Why the two-folder approach?

Without it

- Manual drag-and-drop after each run
- Forget one table ⇒ wrong numbers in paper
- `writing/` and `output/` silently diverge
- Replicator has no clear path from code to paper

With `refresh_output.sh`

- One command re-links every table and figure
- Fully reproducible: `make.sh` → refresh → `pdflatex` = whole paper
- LaTeX always reads the latest output
- `writing/` can live on Overleaf or a separate git branch

Outline 3

1. Background
2. Repository Structure
3. The Scalars Pattern
4. End-to-End Workflow

What are scalars?

Numbers that appear **inline in the text** of your paper:

> *"The cartel raised prices by* ***\PriceEffect****%."*

> *"We observe* ***\NSampleFirms*** *firms and* ***\NTenders*** *tenders."*

If these are hard-coded, any data change forces manual updates across the paper — and errors creep in.

The scalars pattern

Write every such number into `scalars.tex` as a LaTeX `\newcommand` — **automatically, from your analysis code.**

Scalar best practices

Golden rule

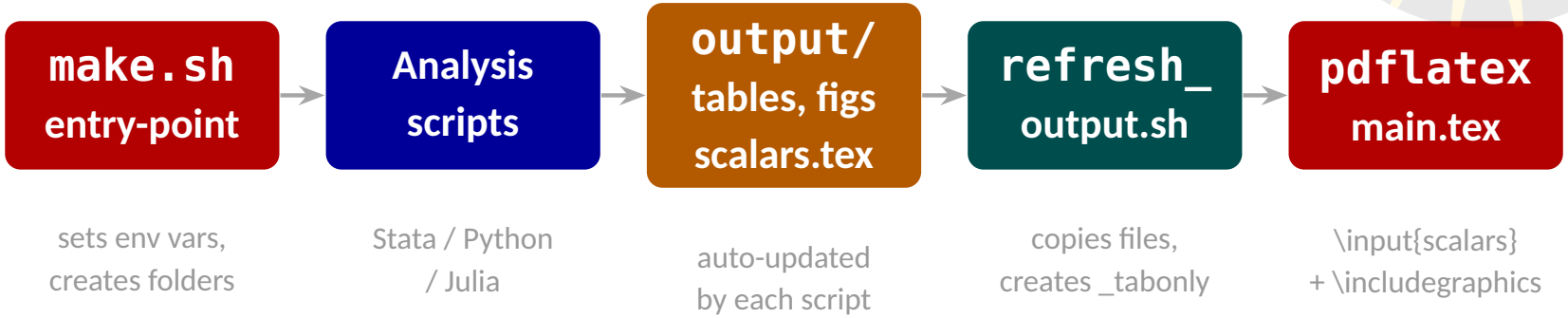
Never hard-code a number in your paper if your code knows it.

(VITA SHOWS)

Outline 4

1. Background
2. Repository Structure
3. The Scalars Pattern
4. End-to-End Workflow

From raw data to published paper — one command



`bash code/make.sh` ⇒ updated, compiled paper. **No manual steps.**

Random issues to take care of

- No absolute path (use config.py or profile.do (VITA SHOWS)
- You need to set seeds
- Have logs
- Draftable (make example) – `https://www.draftable.com/compare` (VITA SHOWS)